

How LLMs Generate Text

End-to-End Inference Pipeline

A Mock Interview Guide

What's Inside

This guide walks you through the *complete* journey of a single forward pass in an LLM — from raw text all the way to the streamed word on your screen. Each of the nine sections opens with a plain-English overview of what's happening and why it matters, then dives into five interview questions with full, structured answers. Whether you're prepping for a machine learning engineering role or just want to genuinely understand how these systems work, you're in the right place. Grab a coffee and let's go!

1. Overview — The Big Picture
2. Input & Tokenizer
3. Embedding Layer
4. Transformer Block
5. KV Cache & Memory
6. LM Head — Logits to Probs
7. Sampling Strategy
8. Speculative Decoding
9. Detokenizer & Streaming

§0 The Big Picture — How a Token Gets Born

The 60-second version.

When you press Enter in ChatGPT, Claude, or any LLM-powered interface, a remarkable chain of events kicks off inside a GPU cluster. Understanding that chain — not just memorising its pieces — is what separates a candidate who can recite facts from one who can *reason* about systems. Here is the story in plain English.

Step 1 — Tokenization. Your message is broken into small fragments called **tokens**. These aren't always whole words; "gravity" might become two tokens: **grav** and **ity**. Each token maps to an integer ID drawn from a fixed vocabulary of roughly 100K–128K entries.

Step 2 — Embedding. Each integer ID is looked up in a giant table called the **embedding matrix**, which returns a dense vector of several thousand numbers. This vector is the model's internal representation of that token. A positional encoding is added on top so the model knows whether a word appeared first or last in the sentence.

Step 3 — The Transformer Stack. The embedding vectors pass through N identical **Transformer layers** (typically 32–96 of them). Inside each layer, **Multi-Head Attention** lets every token look at every other token and gather relevant context. A **Feed-Forward Network** then refines each token's representation independently. Residual connections and Layer Normalisation keep the whole stack stable and trainable.

Step 4 — LM Head. After the final layer, the model has a rich hidden-state vector for the very last token. This is projected through a linear layer (the **LM Head**) into a vector of logits — one per vocabulary entry. A softmax turns these into probabilities.

Step 5 — Sampling. A strategy such as **Top-P** or **Top-K** picks the next token from this distribution. Temperature scales how surprising the choice can be.

Step 6 — Detokenization and Streaming. The chosen token ID is converted back to text and sent to your screen immediately — you see it appear before the model has even started on the next word. Then the whole process repeats, one token at a time, until an end-of-sequence marker appears or a length limit is hit.

Two phases to tattoo on your brain:

- **Prefill** — all input tokens are processed *in parallel* in a single forward pass. This is fast and **compute-bound**. The result is a fully populated KV Cache ready for the decode phase.
- **Decode** — exactly one new token is generated per forward pass, attending to the cached keys and values. This phase is **memory-bound**: the GPU spends most of its time moving data rather than doing arithmetic.

Most of the interesting engineering trade-offs in LLM serving come down to managing these two phases efficiently.

1. Input & Tokenizer

Before a language model can do anything useful, it needs to translate your raw text into a format it can compute with: a sequence of integers. That translation is the job of the **tokenizer**, and it's more nuanced than it sounds.

The tokenizer splits text into **tokens** — small fragments that might be whole words, parts of words, punctuation marks, or even individual bytes. Each token maps to a unique integer ID in a fixed **vocabulary**, typically containing 100,000 to 128,000 entries in modern models. The vocabulary is built once during training using an algorithm like **Byte-Pair Encoding (BPE)**, which learns which character sequences appear together often enough to deserve their own dedicated slot.

The size and design of the vocabulary has real consequences. A larger vocabulary means each token covers more text, so sequences are shorter and decoding is faster. But it also makes the embedding matrix and the final LM Head layer larger, increasing memory usage. Getting this balance right is one of the underappreciated engineering decisions in building a new LLM.

Tokenizers also handle special structural tokens — entries like `<|im_start|>` or `<|endoftext|>` that signal message boundaries, end of generation, and padding. The chat template that wraps your system prompt and user message is enforced entirely through these special tokens. Understanding the tokenizer is understanding the first place where things can go subtly wrong in production.

Q1.1 What exactly is tokenization, and why do modern LLMs use sub-word schemes?

Think of tokenization as the model's way of reading — it needs to break raw text into bite-sized chunks it has seen before. The three main options are:

- **Character-level** — simple, but sequences get very long and the model must learn spelling from scratch. Painful.
- **Word-level** — nicer to read, but the vocabulary explodes into millions of entries and rare words never appear in training. Ouch.
- **Sub-word (BPE, WordPiece, Unigram)** — the sweet spot. Common words stay whole; rare words split into known pieces. Vocabulary stays manageable at around 32K–128K tokens. GPT, LLaMA, Gemma, and friends all use this approach.

BPE in one sentence: repeatedly find the most frequent adjacent pair of character sequences and merge them into a new token — until you hit your target vocabulary size. That's it. Everything else is bookkeeping.

Q1.2 How large is a typical vocabulary, and why does that number actually matter?

Modern LLMs sit around **100K–128K tokens**. GPT-4 uses 100K via the `cl100k_base` tokenizer; LLaMA-3 uses 128K.

Why does it matter? Because the final layer of the model — the LM Head — is a matrix of size $d_{\text{model}} \times V$. At $V = 128\text{K}$ and $d_{\text{model}} = 4096$, that's already roughly half a billion

parameters just for that one layer. A too-large vocabulary wastes memory on tokens that almost never appear; a too-small one forces non-English languages into embarrassingly long sequences, slowing down both training and inference.

Q1.3 What is the difference between a *token* and a *token ID*?

Super simple, and interviewers love seeing you nail this cleanly.

A **token** is a string fragment — "grav", " is", "?". A **token ID** is its row number in the vocabulary lookup table (e.g., 26110).

The tokenizer keeps two dictionaries: **encode** maps a string to an integer (used at *input* time), and **decode** maps an integer back to a string (used by the detokenizer at *output* time). That really is all there is to it!

Q1.4 What is the context window, and what happens if you go over it?

The **context window** is the maximum total number of tokens — input plus generated output — the model can attend to in one go. Think of it as the model's working memory. Modern models range from 8K (older GPT-4) all the way to 2M tokens (Gemini 1.5 Pro).

Hit the limit and you have three options:

- **Truncate** — chop off the oldest tokens. Simple but lossy.
- **Chunk and retrieve** — RAG-style: break the document into pieces and only fetch what's relevant.
- **Extend the architecture** — RoPE scaling, YaRN, or sliding-window attention can push the effective length without full retraining, though quality often degrades at extreme lengths.

Q1.5 What are special tokens, and could you give a few examples?

Special tokens are reserved vocabulary slots that carry *structure*, not meaning. They're the punctuation of the prompt format.

Token	What it signals
< endoftext >	End of document — the model should stop generating
< im_start >	Start of a chat message (ChatML format)
< im_end >	End of a chat message
<pad>	Filler to make sequences the same length in a batch

2. Embedding Layer

Once the tokenizer has produced a sequence of integer IDs, the model needs to convert them into something it can actually compute with: dense, continuous vectors. This is the job of the **embedding layer**.

The embedding layer is essentially a giant lookup table called the **embedding matrix** $E \in \mathbb{R}^{V \times d_{\text{model}}}$, where V is the vocabulary size and d_{model} is the model's hidden dimension. To embed token ID i , you simply grab row i of this matrix. The result is a vector of d_{model} numbers — 4096 for a 7B-class model, 8192 for a 70B-class model — that encodes the semantic and syntactic identity of that token as the model has learned it during training.

But there is a subtle problem: raw token embeddings are **position-agnostic**. If you shuffle the words in a sentence, the embedding vectors are identical — the model has no way to know which word came first. That's where **positional encodings** come in. They inject information about each token's position in the sequence into its embedding vector, either by adding a fixed mathematical pattern (sinusoidal encoding) or by rotating the query and key vectors during attention (RoPE, which nearly all modern models use).

The embedding layer is also where a popular memory-saving trick lives: **weight tying**. Because the input embedding matrix and the final output projection (the LM Head) are both of shape $V \times d_{\text{model}}$, many models simply share the same weights between them. This halves the number of embedding parameters at minimal quality cost, which matters a lot when your vocabulary has 128,000 entries.

Q 2.1 What is the embedding matrix and what does it actually do?

You can think of the embedding matrix $E \in \mathbb{R}^{V \times d_{\text{model}}}$ as a **giant cheat sheet**. Each row is a learned, dense vector for one token. To embed token ID i , you grab row i :

$$\mathbf{e}_i = E[i, :] \in \mathbb{R}^{d_{\text{model}}}$$

That's a lookup, not a matrix multiply. For GPT-4-class models, $d_{\text{model}} \approx 4096$, so each token becomes a 4096-dimensional vector. The model learns during training which directions in that space mean “noun”, “negative sentiment”, “French-language”, and so on.

Q 2.2 Why do we need positional encodings on top of embeddings?

Here's a fun thought experiment: if you shuffle the words in a sentence, the embeddings are identical — the model has no idea word 1 came before word 2. **Positional encodings** inject sequence-order information.

The main flavours today:

- **Sinusoidal** — the original from “Attention is All You Need”. Fixed sin/cos patterns, no extra parameters, doesn't extrapolate perfectly.
- **Learned absolute** — an extra embedding table, one per position. Simple but doesn't generalise well beyond training length.
- **RoPE (Rotary Position Embedding)** — rotates the query and key vectors based on their relative positions. Dominant in LLaMA, Mistral, GPT-4, and most modern models.

Extrapolates naturally and is compatible with KV Cache.

- **ALiBi** — adds a linear distance bias to attention scores. No extra parameters, great length generalisation.

Q 2.3 What are the dimensions of the embedding matrix and why does d_{model} matter so much?

Every weight matrix in the Transformer is tied to d_{model} . The attention projections are $d_m \times d_m$; the FFN inner layer is $4d_m$ wide. Change d_m and the whole model scales with it. Typical values: 4096 for 7B models, 8192 for 70B models. The embedding matrix alone at $128\text{K vocab} \times 4096 = \text{approximately } 0.5\text{B parameters}$. That's why weight tying (sharing it with the output LM Head) is such a popular trick — it halves the embedding footprint with minimal quality trade-off.

Q 2.4 What is weight tying and why is it used?

The input embedding matrix maps tokens *in*, and the LM Head maps hidden states back *out* to vocabulary logits. Their shapes are $V \times d_m$ and $d_m \times V$ — they are transposes of each other.

Weight tying literally sets $W_{\text{out}} = E^T$, halving the number of embedding parameters. The intuition is elegant: if two tokens are “nearby” in the input embedding space, they should also compete closely in the output distribution — and tying forces that consistency across both directions.

Q 2.5 What happens to embeddings of tokens that barely appear in training data?

They end up **poorly trained** — gradients have barely touched those rows, so they stay near their random initialisation. In practice:

- Rare tokens tend to cluster near the origin with small magnitude.
- They may share embedding space chaotically with unrelated tokens.
- This is one reason larger vocabularies can hurt on low-resource languages — too many tokens get too little training signal.

Techniques like **subword regularisation** and **BPE-dropout** deliberately vary tokenization during training so rare segments get more exposure.

3. Transformer Block

The Transformer block is the core computational unit of the model, and it is stacked N times (32 in a 7B model, 96 in a GPT-4-scale model). Every block runs two sub-layers in sequence: **Multi-Head Self-Attention** and a **Feed-Forward Network (FFN)**, with a residual connection and a LayerNorm wrapping each one.

Why attention? Before Transformers, language models processed text left to right through recurrent networks, which meant the signal from an early word had to survive dozens of timesteps to influence a later one — a notorious source of vanishing gradients. Attention sidesteps this entirely: every token directly computes a relevance score with every other token in the sequence, allowing long-range dependencies to be captured in a single layer.

Scaled dot-product attention is the mechanism that computes those scores. Each token's embedding is linearly projected into three vectors: a Query (“what am I looking for?”), a Key (“what do I contain?”), and a Value (“what should I pass on?”). Queries dot-product with Keys to produce attention weights, which then form a weighted average of the Values. Crucially, a **causal mask** prevents each token from attending to tokens that haven't been generated yet, enforcing the autoregressive property.

Multi-Head Attention runs this mechanism in parallel across several independent “heads”, each operating in a lower-dimensional subspace. Different heads specialise in different relationships — syntax, coreference, positional patterns — and their outputs are concatenated and projected back to d_{model} .

The Feed-Forward Network processes each position independently with a two-layer MLP. Despite its simplicity, this sublayer is where much of the model's factual knowledge is stored. Neurons in the FFN have been shown to act as key-value memories that activate for specific concepts and facts encountered during training.

Residual connections and LayerNorm are the glue that makes training 96-layer networks feasible. Without residuals, gradients would vanish to zero before reaching the first layer. With them, there is always a direct path from the loss to early weights, no matter how deep the network.

Q3.1 Walk me through scaled dot-product attention — formula and intuition.

Here's the formula, and then let's make it feel natural:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

The intuition: queries are “what am I looking for?”, keys are “what do I have?”, and values are “here's the actual content”. The dot product $QK^\top$ scores how well each query matches each key. Softmax turns those scores into weights that sum to 1. Then we take a weighted average of the values.

Why divide by $\sqrt{d_k}$? When d_k is large, dot products blow up in magnitude, pushing softmax into saturation where gradients vanish. Dividing by $\sqrt{d_k}$ keeps variance at roughly 1 regardless of dimension.

Q3.2 What does Multi-Head Attention add over single-head attention?

Running one attention function gives you one *way* of relating tokens. MHA runs h heads in parallel, each in its own lower-dimensional subspace:

$$\text{head}_i = \text{Attn}(QW_i^Q, KW_i^K, VW_i^V) \quad \text{MHA} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Each head can specialise in a different type of relationship — syntactic agreement, coreference, positional patterns. The concatenated output is projected back to d_{model} . The key trick: $d_k = d_{\text{model}}/h$, so total compute is the same whether you use 1 head or 32.

Q 3.3 What is the Feed-Forward Network and what does it actually do?

After attention routes information *between* positions, the FFN processes each position *independently*:

$$\text{FFN}(\mathbf{x}) = W_2 \sigma(W_1 \mathbf{x} + b_1) + b_2$$

The hidden dimension is typically $4d_{\text{model}}$ (or $8d_m/3$ with SwiGLU activation). Research has shown that FFN neurons act like **key-value memories**: individual neurons fire for specific facts and concepts encountered in training. Attention finds the right context; the FFN retrieves the right knowledge. A tidy division of labour.

Q 3.4 What are residual connections, and why are they so important?

Each sub-layer wraps its output in a **residual (skip) connection**:

$$\mathbf{x} \leftarrow \mathbf{x} + \text{SubLayer}(\text{LayerNorm}(\mathbf{x}))$$

Without residuals, gradients must multiply through every layer on their way back to layer 1 — 96 multiplications, and the signal vanishes to nearly zero. The residual path is a **gradient highway** that lets loss signals reach early layers directly. It also means a poorly initialised sub-layer can't crash the whole network: the residual carries the original signal forward regardless.

Q 3.5 Why is LayerNorm applied *before* each sub-layer in modern models?

The original paper applied LayerNorm *after* (Post-LN). Modern models use **Pre-LN** — normalise before each sub-layer:

- **Training stability**: the sub-layer input is always normalised, preventing gradient explosions deep in the network.
- **Higher learning rates**: Pre-LN models converge more reliably with aggressive optimisa-

tion schedules.

- **Better final-layer behaviour:** in Post-LN, the last layer's output is un-normalised, causing instability.

GPT-2 pioneered Pre-LN and every mainstream model since has followed.

4. KV Cache & Memory

During autoregressive decoding, the model generates one token at a time. To compute attention for the new token, it needs the key and value vectors for *every* previous token. Without any caching, this would require recomputing those vectors from scratch on every step — scaling as $O(T^2)$ over the sequence length. The **KV Cache** eliminates that redundancy by storing the key and value matrices from every Transformer layer for all tokens seen so far. Each new decode step only computes Q, K, and V for the one new token, appends the new K and V to the cache, and reads the rest from memory.

This sounds like a pure win, and it mostly is — but it comes with a serious cost. The KV Cache grows linearly with sequence length and the number of layers. For a 70B-parameter model processing a 4K-token sequence in FP16, the cache alone consumes roughly **10 GB of GPU memory**. Multiply that across a batch of concurrent users, and you understand quickly why serving large models at scale is a memory engineering problem far more than a compute problem.

Three techniques have emerged to manage this: **FlashAttention** avoids materialising the full attention score matrix in slow GPU memory by computing it in tiles using fast on-chip cache. **Grouped-Query Attention (GQA)** shares a single set of key and value heads across multiple query heads, shrinking the cache by a factor equal to the grouping ratio — LLaMA-3, Mistral, and Gemma all use this. **KV Cache quantisation** stores the cached tensors in INT8 or INT4 rather than FP16, halving or quartering the memory footprint with minimal quality loss.

Understanding the KV Cache is, in practice, the most important single concept for anyone building or scaling LLM inference systems.

Q4.1 What is the KV Cache and why is it essential?

Without caching, every time we generate token $t + 1$, we'd recompute the *entire* attention for all t previous tokens — $O(T^2)$ compute per step. The **KV Cache** stores the key and value matrices from every layer for all tokens seen so far.

When generating token $t + 1$, we only need to:

1. Compute Q, K, V for the *one new token*.
2. Read cached K, V (no recomputation).
3. Append the new K, V to the cache.

Each decode step is now effectively $O(1)$ compute. This is what makes streaming responses feel fast rather than waiting minutes for a full output.

Q4.2 How much GPU memory does the KV Cache actually use?

Let's walk through it with real numbers — this is a classic interview deep-dive:

$$\text{KV size (bytes)} = 2 \times N_L \times T \times d_m \times \text{dtype_bytes}$$

For a 70B model (FP16): $N_L=80$ layers, $d_m=8192$, $T=4096$ tokens, 2 bytes per value:

$$2 \times 80 \times 4096 \times 8192 \times 2 \approx \mathbf{10.7 \text{ GB}}$$

That's just the cache — before loading model weights! This is why long-context inference is

so memory-hungry, and why GQA and quantisation are so critical in production serving.

Q 4.3 What is FlashAttention and how does it help?

Standard attention materialises the full $T \times T$ score matrix in GPU **HBM** (the main off-chip memory). For $T = 8192$, that's roughly 128 MB per layer per batch. Memory bandwidth is slow.

FlashAttention rewrites the attention kernel to process the matrix in small **tiles** that fit in fast on-chip SRAM, fusing softmax and the matrix multiply into a single pass. It *never* writes the full score matrix to HBM:

- HBM reads/writes drop from $O(T^2)$ to $O(T)$.
- Memory usage is drastically reduced.
- Speed-up: 2–4× on A100/H100.

FlashAttention-2 and 3 push further with better thread-level parallelism.

Q 4.4 What is Grouped-Query Attention (GQA) and why does it matter?

Standard MHA has h query heads, h key heads, and h value heads — so the KV Cache grows with h . **GQA** shares K and V heads across groups of query heads:

- **MHA**: h KV heads — maximum expressiveness, maximum cache.
- **GQA**: h/g KV heads shared across g query groups — cache shrinks by factor g , minimal quality loss.
- **MQA**: 1 KV head for all queries — minimum cache, some quality trade-off.

LLaMA 3, Mistral, and Gemma all use GQA, allowing 70B models to fit on fewer GPUs without compromising output quality meaningfully.

Q 4.5 Why is decode *memory-bound* rather than *compute-bound*?

During **prefill**, all T input tokens are processed together — lots of arithmetic per byte loaded. GPU compute units stay busy. **Compute-bound**.

During **decode**, only *one* token is processed per step. The GPU loads the full KV Cache from HBM, does a tiny amount of arithmetic, then waits for the next memory load. Compute units sit idle. **Memory-bound**.

This means faster GPU memory (HBM3e on H100), smaller caches (GQA, INT8 quantisation), and batching more requests together are the main levers for improving decode throughput — not higher raw FLOP counts.

5. LM Head — From Hidden State to Probability

After the final Transformer layer, the model has processed all context and distilled it into a single hidden-state vector $\mathbf{h} \in \mathbb{R}^{d_{\text{model}}}$ representing what it “thinks” should come next. The job of the **LM Head** is to translate that abstract representation into a concrete **probability distribution over the entire vocabulary**.

It does this in two steps. First, a linear projection (a simple matrix multiply, no activation function) maps $\mathbf{h}$ to a vector of **logits** $\ell \in \mathbb{R}^V$, where V is the vocabulary size. Each logit is the model’s unnormalised log-probability for the corresponding token being next. Second, a **softmax** function normalises these logits into proper probabilities that sum to 1.

The **temperature** parameter is applied here, dividing all logits by a scalar T before the softmax. A temperature near zero collapses the distribution to the single most likely token (greedy decoding). A temperature above 1 flattens the distribution, making surprising tokens more likely. In practice, most chat deployments use $T \approx 0.7\text{--}1.0$, trading a small amount of coherence for noticeably more natural-sounding variety.

The LM Head is computationally significant. At $d_m = 4096$ and $V = 128\text{K}$, the weight matrix alone has 524 million parameters — and it must be multiplied at every single decode step. **Weight tying** (sharing this matrix with the input embedding) is the dominant trick for reducing this cost, and it works so well that nearly every modern LLM does it.

Q 5.1 What does the LM Head do, and why is it a linear projection?

After the last Transformer layer, the hidden state $\mathbf{h} \in \mathbb{R}^{d_m}$ is projected into **logits**:

$$\ell = W_{\text{out}} \mathbf{h} \in \mathbb{R}^V$$

It is *linear* (no activation) because softmax is the nonlinearity that follows. The raw logits are the model’s unnormalised log-probabilities — larger means more likely.

Q 5.2 What is temperature and how does it change the model’s behaviour?

Temperature T rescales logits before softmax:

$$p_i = \frac{e^{\ell_i/T}}{\sum_j e^{\ell_j/T}}$$

Think of it as a **creativity dial**:

- $T \rightarrow 0$: collapses to argmax — always the most likely token. Deterministic. Great for factual answers.
- $T = 1$: the raw training distribution. Balanced.
- $T > 1$: flattens the distribution — more surprising, more creative, also more likely to go off-script.

Chat models typically use $T \approx 0.7\text{--}1.0$. Code generation often uses lower values for precision.

Q 5.3 Why is *log-softmax* preferred over plain softmax during training?

When logit values are large, e^{ℓ_i} overflows to infinity in floating point. **Log-softmax** avoids this with the log-sum-exp trick:

$$\log p_i = \ell_i - \log \sum_j e^{\ell_j} \quad (\text{subtract } \max(\ell) \text{ inside the sum for stability})$$

PyTorch's `F.cross_entropy` does this automatically and fuses it with the NLL loss into a single numerically stable kernel. You almost never want to call `softmax` then `log` then `nll_loss` separately.

Q 5.4 What is the computational cost of the LM Head for large vocabularies?

The LM Head is a matmul of shape $(B \cdot T) \times d_m$ against $d_m \times V$. At $d_m = 4096$ and $V = 128K$:

$$128,000 \times 4,096 \approx 0.5\text{B parameters in one layer}$$

That single operation is one of the most expensive in every forward pass. Motivation for:

- **Weight tying** (shares memory with the input embedding).
- **Vocabulary pruning** (remove tokens unseen in the fine-tuning domain).
- **Speculative decoding** (amortise LM Head cost across multiple draft tokens verified in one pass).

Q 5.5 What is Top-K logit masking and how does it improve sampling stability?

After computing logits but *before* softmax, we set all logits except the K largest to $-\infty$. This prevents the model from ever sampling extremely low-probability nonsense tokens lurking in the long tail.

Setting $K = 1$ recovers greedy decoding. A typical production value is $K = 50$. Top-K is often combined with **Top-P** (nucleus sampling): first keep the top 50 tokens, then further trim to those whose cumulative probability exceeds p . You get the best of both worlds — a hard cap and an adaptive cutoff.

6. Sampling Strategy

At this point in the pipeline, the model has done all of its expensive computation and produced a probability distribution over the vocabulary. The sampling strategy is what decides which token to actually *pick* from that distribution.

This choice matters more than it might seem. Pick the single most likely token every time (**greedy decoding**) and the output is coherent but repetitive and bland — the model always says the safe, average thing. Allow too much randomness and the output becomes incoherent. A good sampling strategy threads the needle between predictability and creativity.

Top-K sampling restricts the sampling pool to the K most likely tokens, discarding the long tail of unlikely choices. This works well, but K is a fixed number that doesn't adapt to the model's confidence. When the model is very sure (one token has 90% probability), a K of 50 still allows 49 bad options. When the model is genuinely uncertain, $K = 50$ might cut off legitimate alternatives.

Top-P sampling (nucleus sampling) solves this adaptively: keep the smallest set of tokens whose cumulative probability exceeds a threshold p (typically 0.9 or 0.95). When the model is confident, the nucleus is small. When uncertain, it grows naturally. This is why Top-P has largely supplanted Top-K in production chat systems.

Temperature, applied before softmax, is the global creativity dial that interacts with both methods. **Repetition penalty** discounts already-generated tokens to prevent the model from looping. **Constrained decoding** goes further, restricting the sample space to only tokens that keep the output syntactically valid according to a grammar or JSON schema.

Q6.1 Compare Greedy, Top-K, and Top-P sampling.

Method	How it works	Best for
Greedy	Always pick argmax	Factual QA, SQL, exact extraction
Top-K	Sample from top K tokens	Code generation, structured output
Top-P	Sample from nucleus $\geq p$	Chat, story writing, open-ended tasks

The advantage of **Top-P** over Top-K: when the model is very confident (peaky distribution), only a few tokens pass the threshold. When uncertain (flat distribution), many are allowed. It *adapts* to the model's confidence, which Top-K cannot.

Q6.2 What is repetition penalty and when is it useful?

Left unchecked, models love to repeat themselves. If “gravity” was just generated, it has a high probability score — so the model wants to say it again. **Repetition penalty** discounts already-seen tokens:

$$\ell'_i = \ell_i / r \quad \text{for any token } i \text{ seen in context} \quad (r > 1)$$

$r = 1.0$: no effect. $r = 1.3$: gentle. $r = 1.5$: strong. Be careful — set it too high and the model starts avoiding perfectly correct vocabulary, producing incoherent output.

Q 6.3 What is beam search and how does it differ from sampling?

Beam search keeps B candidate sequences alive simultaneously. At each step it expands every beam to the top- B tokens and prunes back to the B highest log-probability full sequences.

- **Pro**: finds high-probability sequences; reproducible.
- **Con**: tends toward bland, generic, repetitive output. The highest-probability sentence is often the most boring one.

Still popular in **machine translation** and **summarisation** where fidelity matters more than creativity. For chat and creative writing, stochastic sampling almost always wins on human preference evaluations.

Q 6.4 What are logit processors and how are they used in practice?

A **logit processor** is a callable that transforms raw logits before sampling. They chain in a pipeline:

- **TemperatureLogitsWarper** — divides by T .
- **TopKLogitsWarper** — masks everything below top- K .
- **TopPLogitsWarper** — applies nucleus masking.
- **RepetitionPenaltyLogitsProcessor** — discounts repeated tokens.
- **StructuredOutputProcessor** — forces valid JSON/schema tokens only.

HuggingFace Transformers and vLLM use this pattern. The beauty is composability — mix and match processors without touching the model code.

Q 6.5 What is constrained decoding, and give a real use case?

Instead of freely sampling from the whole vocabulary, at each step you **mask out any token that would make the output invalid** according to a grammar or schema.

Example: to guarantee valid JSON, a finite-state machine (FSM) tracks the parse state. If the FSM says “we’re inside a string value”, only tokens that are valid string content get non- $-\infty$ logits. Closing braces are only allowed when the FSM is in a valid terminal state. Libraries like **Outlines**, **lm-format-enforcer**, and **Guidance** implement this. It is especially useful for **function calling**, **SQL generation**, and any tool-use scenario where the output must be parsed by downstream code.

7. Speculative Decoding

Speculative decoding is one of the most elegant engineering ideas in modern LLM serving, and it delivers $2\text{--}3\times$ throughput improvement with *mathematically guaranteed* zero change in output quality.

The key insight is asymmetry in cost: **verifying** k tokens costs roughly the same as generating a single one, because the large target model can process all k positions in parallel during a single forward pass. This means that if a faster, smaller model can generate a plausible sequence of k candidate tokens, the large model can verify all of them in one shot and accept the ones that match its distribution.

The algorithm works as follows. A **draft model** (typically $5\text{--}10\times$ smaller than the target) autoregressively produces k candidate tokens. The **target model** then runs one forward pass over the original context plus all k candidates, producing $k + 1$ logit vectors simultaneously. Each draft token is accepted with probability $\min(1, p/q)$ where p is the target's probability and q is the draft's probability. On the first rejection, a correction token is sampled from the adjusted distribution $(p - q)_+$, which preserves the exact target distribution in expectation.

The acceptance rate depends on how well the draft model's distribution matches the target's. A well-matched pair (same family, same fine-tuning domain) achieves acceptance rates of $0.7\text{--}0.9$. Even at lower acceptance rates, the throughput usually improves because the rare full rejection is still faster than always generating one token at a time.

Beyond the two-model setup, newer approaches like **Medusa** and **EAGLE** attach lightweight prediction heads directly to the main model, eliminating the need to maintain a separate checkpoint entirely.

Q7.1 Walk me through speculative decoding step by step.

The insight: *verifying k tokens costs roughly the same as generating 1.*

1. **Draft**: a small, fast model autoregressively generates k candidate tokens (typically $k = 4\text{--}8$).
2. **Verify**: the large target model runs *one* forward pass on the original context + all k drafts simultaneously, producing $k + 1$ logit vectors at once.
3. **Accept/Reject**: for each draft token x_i , accept with probability $\min(1, p(x_i)/q(x_i))$.
4. **Correct**: at the first rejection, sample from $(p - q)_+$, ensuring the output matches the target distribution exactly.

Net result: $2\text{--}3\times$ throughput with *zero* quality degradation.

Q7.2 What mathematical guarantee does speculative decoding provide?

The accept/reject step is an instance of **rejection sampling**. The result is that the distribution of accepted tokens is *identical* to sampling directly from p :

Accept x with probability $\min\left(1, \frac{p(x)}{q(x)}\right)$, then sample correction from $\max(0, p - q)$ on rejection.

The key theorem: the marginal distribution over the output sequence is exactly p , regardless of how bad the draft model is. A terrible draft just lowers the acceptance rate — it *never* corrupts output quality.

Q 7.3 What makes a good draft model?

The acceptance rate α drives everything. To maximise α :

- **Same family and training domain** — if the draft was fine-tuned on different data, its token preferences diverge from the target's, killing acceptance rate.
- **5–10× smaller** — big enough to draft well, small enough to be genuinely fast.
- In practice, acceptance rates of **0.7–0.9** are achievable with a well-matched pair (e.g. LLaMA-3-8B drafting for LLaMA-3-70B).

Q 7.4 Are there versions that don't need a separate draft model?

Yes — maintaining two checkpoints is operationally awkward. Alternatives:

- **Medusa**: train k extra linear heads atop the main model, each predicting tokens $t+1, \dots, t+k$ simultaneously.
- **EAGLE**: a lightweight auxiliary model that operates on the main model's hidden states, achieving higher acceptance rates than a naive small model.
- **Self-speculative (early exit)**: intermediate layers act as the draft.

Medusa and EAGLE are the most production-friendly options today.

Q 7.5 How does speculative decoding interact with the KV Cache?

When draft tokens are *rejected*, the KV Cache must be **rolled back**: keys and values for rejected tokens were appended but must be invalidated.

Production serving systems (vLLM, TensorRT-LLM) handle this with **paged KV caches** — memory is allocated in small pages, so invalidating rejected tokens means freeing those pages rather than copying large contiguous blocks. If j out of k tokens are accepted, the cache grows by j entries. This bookkeeping is the main engineering complexity of speculative decoding at scale.

8. Detokenizer & Streaming Output

The decode loop produces a stream of integer token IDs. The final job of the inference pipeline is to convert those IDs back into human-readable text and deliver them to the user as fast as possible. This is the work of the **detokenizer** and the **streaming layer**.

The detokenizer is the inverse of the tokenizer: it looks up each integer in the vocabulary table and concatenates the resulting strings. But this is trickier than it sounds for **byte-level BPE** models (such as GPT-4), where tokens may represent raw bytes rather than complete Unicode characters. A single token might be the first byte of a multi-byte UTF-8 sequence, and the detokenizer must buffer incomplete bytes until a full character can be formed. Releasing a partial byte to the user would produce a garbled character — a bug that is surprisingly easy to introduce.

Streaming is what makes the user experience feel interactive. Rather than waiting for the entire response to be generated before sending anything, the server sends each detokenized token the moment it is produced using **Server-Sent Events (SSE)**, a simple HTTP streaming protocol. The client appends each chunk to its display buffer, creating the impression that the model is typing in real time.

This shift from batch delivery to streaming has a profound effect on **perceived latency**. The metric that users feel is not total generation time but **TTFT (Time To First Token)** — the delay from submitting the request to seeing the first word appear. TTFT corresponds to the prefill phase. A 500-token response at 50 tokens per second takes 10 seconds to complete, but if the first token arrives in 200 milliseconds, the experience feels instant.

Finally, the generation must **terminate** — either because the model produces an end-of-sequence token, the output hits a configured length limit, or a stop sequence specified by the user appears in the decoded text. Getting termination right in streaming systems requires careful attention because stop sequences must be detected in the *decoded* text, not the raw token IDs, since a string like "###" may map to different token boundaries in different contexts.

Q8.1 How does the detokenizer reconstruct text, and what are the tricky edge cases?

The detokenizer looks up each integer ID in the vocabulary and concatenates the strings. A few edge cases trip people up:

- **Byte-level BPE**: tokens may be raw bytes, not full Unicode characters. The decoder must buffer bytes until a complete character forms.
- **Leading spaces**: " is" (space + is) is one token in BPE. These spaces must be preserved for correct word boundaries.
- **Special tokens**: `<|endoftext|>` and `<|im_end|>` must be stripped, not forwarded to the user.

Q8.2 What is streaming output and how is it implemented at the API level?

Without streaming, you'd stare at a blank screen for 10 seconds and then see the whole answer pop in at once. Terrible UX.

Streaming delivers each token the moment it's generated via **Server-Sent Events (SSE)** — the server sends a new HTTP chunk per token, structured as a JSON delta. The client

appends each chunk to its display. The user sees tokens appear word by word, as if the model is typing.

Why this matters: perceived latency drops from *total generation time* to **TTFT (Time To First Token)** — the prefill phase duration only. For a 500-token response, that difference is enormous.

Q8.3 What are TTFT and TPS, and why do engineering teams obsess over them?

These are the two **Service Level Objectives (SLOs)** for LLM serving:

Metric	Definition	Optimised by
TTFT	Time until first token	Fast prefill, H100, short prompts
TPS	Tokens per second generated	KV Cache efficiency, GQA, quantisation
E2E latency	$\text{TTFT} + \text{output_len} / \text{TPS}$	Both

Typical targets: TTFT < 500ms, TPS > 50 for interactive chat.

Q8.4 What are stop sequences and how does generation terminate?

Decoding ends when any of these occur — checked after each token:

1. **EOS token**: the model generates `<|endoftext|>` or `<|im_end|>`.
 2. **max_tokens**: output length hits the configured limit.
 3. **Stop sequence**: a user-specified string appears in the decoded output (e.g. `"\n--\n"`).
- Stop sequences are matched against decoded text, not token IDs, because a string like `"###"` may split into different tokens in different contexts. That subtlety bites people in production.

Q8.5 What is continuous batching and why does it matter for throughput?

Old-school **static batching**: pad all sequences in a batch to the length of the longest one. Every request must finish before a new batch starts. GPU sits idle waiting for the slowest sequence. Terrible utilisation.

Continuous batching (in-flight batching):

- At every decode step, new requests join and completed ones leave.
- No padding waste — every GPU cycle is used for real tokens.
- vLLM's **PagedAttention** implements this with a paged KV Cache memory system (borrowed from OS virtual memory).

This is the single biggest practical throughput win in LLM serving — often a 10–20× im-

provement over static batching.

§ Quick Reference — Key Numbers to Know Cold

Concept	Typical Value	Why it matters
Vocabulary size	100K-128K	Sets LM Head size and sequence length
Model dimension d_m	4096-8192	Scales all weight matrices quadratically
Attention heads	32-96	MHA parallelism; $d_k = d_m/h$
Num layers N	32-96	Depth; more = capacity + memory
FFN multiplier	4x (SwiGLU: 8x/3)	Inner dimension of feed-forward layer
Context length	8K-2M tokens	Determines KV Cache size
Temperature	0.7-1.0	Creativity dial; 0 = greedy
Top-P threshold	0.90	Nucleus sampling cutoff
Top-K	50	Hard token limit before nucleus
KV Cache formula	$2NTd_m$ bytes	Grows linearly with sequence length
Speculative k	4-8 tokens	Draft length per verification pass
TTFT target	< 500ms	Perceived response latency
TPS target	> 50 tok/s	Chat feels instant above this

Friendly Reminders Before Your Interview

- **Prefill vs Decode** is the single most important mental model. Know it cold: compute-bound vs memory-bound, parallel vs sequential. Almost every systems question leads back here.
- The **KV Cache** formula will come up. Be ready to estimate sizes for a 70B model on the fly — $2 \times 80 \times T \times 8192 \times 2$ bytes. Practice saying that out loud.
- **Scaled dot-product attention**: be able to derive *why* $\sqrt{d_k}$ matters. It is a 30-second argument about variance and gradient stability. Know it cold.
- **Top-K vs Top-P**: don't just memorise the names. Be ready to explain *why* Top-P is adaptive and when that matters more than a fixed cutoff.
- **Speculative decoding** is a favourite systems question. The accept/reject mechanism is mathematically beautiful — understand why it guarantees zero quality loss, not just that it does.
- **FlashAttention, GQA, quantisation** are the three main efficiency levers. Know one real number for each: FlashAttention gives 2-4x speedup; GQA reduces cache by factor g ; INT8 halves cache size.
- Most importantly — **enjoy the conversation!** Interviewers love candidates who are

genuinely excited about how this stuff works. Curiosity is contagious.

Good luck — you have got this.